



Fișă de lucru – Operatori specifici și validarea colecțiilor de date în Python

Noțiuni teoretice extinse

1. Ce sunt operatorii specifici ai listelor și cum funcționează?

- **Operatori de colecție:** Spre deosebire de metodele clasei list (care se apelează cu punct, ex: .append()), operatorii sunt simboluri sau cuvinte cheie rezervate (+, *, in) care permit manipularea directă, combinarea, compararea sau verificarea elementelor din structurile de date. Ei returnează fie o listă nouă, fie o valoare booleană (True / False).

2. Tipuri de operatori pentru structuri de date

- **[] (Operatorul de acces)** — Utilizat pentru a accesa direct un singur element din listă pe baza indicelui său pozițional (pozitiv sau negativ).
 - *Exemplu:* Dacă avem lista coduri = [101, 202, 303], expresia coduri[1] va accesa elementul de pe a doua poziție, returnând 202.
- **in (Operatorul de apartenență)** — Verifică dacă o anumită valoare există în interiorul listei. Returnează True dacă elementul este găsit și False în caz contrar.
 - *Exemplu:* Pentru lista valide = [10, 20, 30], expresia 20 in valide va returna True.
- **not in (Operatorul de non-apartenență)** — Verifică dacă o anumită valoare **NU** se află în interiorul listei. Returnează True dacă elementul lipsește și False dacă elementul este prezent.
 - *Exemplu:* Pentru lista valide = [10, 20, 30], expresia 50 not in valide va returna True (deoarece 50 chiar nu face parte din listă).
- **+ (Operatorul de concatenare)** — Permite unirea a două sau mai multe liste într-o structură complet nouă. Spre deosebire de .extend() (sau .append()), care modifică lista inițială pe loc, operatorul + lasă listele originale intacte și generează o a treia listă în memorie.
 - *Exemplu:* Dacă avem L1 = [1, 2] și L2 = [3, 4], expresia L1 + L2 va genera lista nouă [1, 2, 3, 4].
- *** (Operatorul de multiplicare / replicare)** — Multiplică elementele unei liste de un număr specificat de ori, creând o listă nouă în care elementele originale se repetă ciclic.
 - *Exemplu:* Pentru lista baza = [0], expresia baza * 3 va genera o listă nouă formată din trei elemente identice: [0, 0, 0].
- **== și != (Operatorii relaționali / de comparare)** — == verifică dacă două liste sunt perfect identice (au exact aceleași elemente, în aceeași ordine). != verifică dacă listele sunt diferite (fie ca elemente, fie ca ordine).
 - *Exemplu:* Dacă A = [1, 2] și B = [1, 2], atunci A == B returnează True. Dacă C = [2, 1], atunci A == C returnează False (ordinea contează).

Activitate practică:

Scenariu: „Gestiunea Logistică și Validarea Stocurilor din Depozite”

Context: Lucrezi ca dezvoltator de software pentru un lanț de distribuție. Compania gestionează stocurile de produse prin două depozite regionale (Depozitul Nord și Depozitul Sud). Trebuie să combini listele de produse din cele două depozite, să multiplici stocul unui produs esențial, să verifici apartenența unor coduri introduse de utilizatori pentru siguranță și să compari listele pentru auditul final.

✂ *Cerințe :*

Implementați în Python următoarele:

Pasul 1: Inițializarea stocurilor regionale Declarați o listă numită depozit_nord care conține codurile produselor: [101, 102, 105]. Declarați o a doua listă numită depozit_sud care conține codurile: [201, 204].

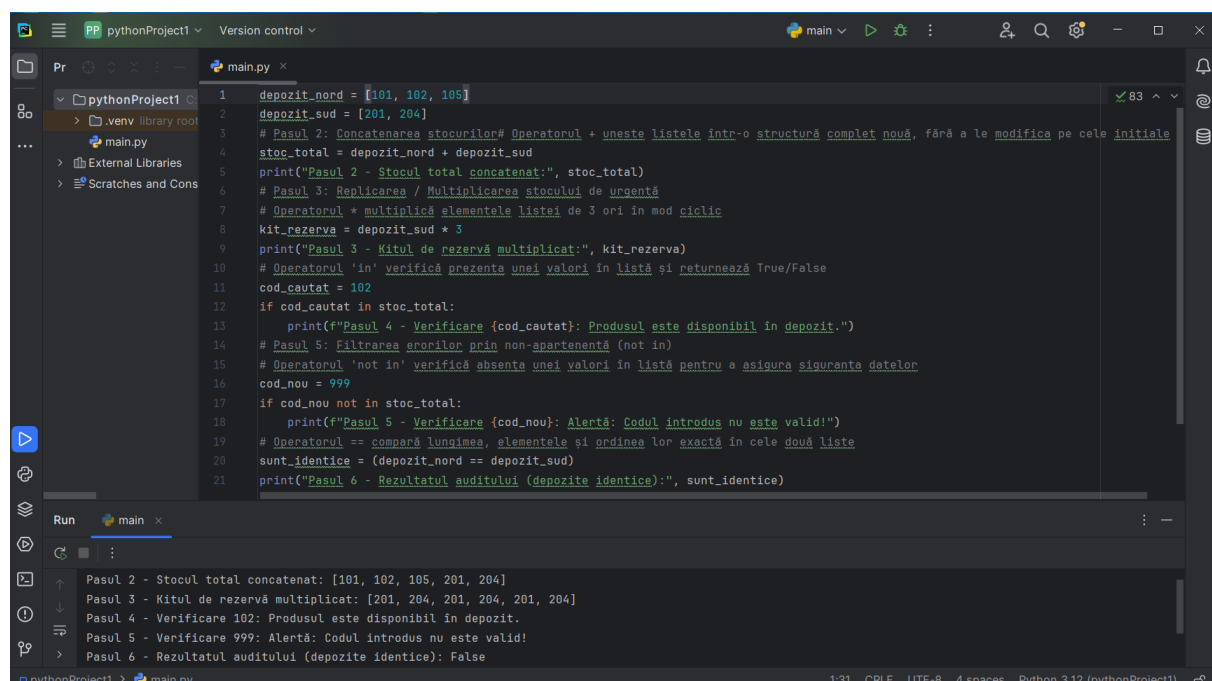
Pasul 2: Concatenarea stocurilor Creați o listă complet nouă, numită stoc_total, prin unirea celor două depozite folosind **operatorul de concatenare (+)**. Afișați lista rezultată pentru a demonstra că stocurile s-au combinat fără a altera listele inițiale.

Pasul 3: Replicarea / Multiplicarea stocului de urgență Depozitul central solicită crearea unui kit de rezervă format din 3 replici ale listei depozit_sud. Generați o listă nouă numită kit_rezerva utilizând **operatorul de multiplicare (*)** aplicat listei din sud. Afișați rezultatul.

Pasul 4: Validarea prin apartenență (in) Un operator de la tastatură introduce un cod pentru verificare: cod_cautat = 102. Folosind o structură condițională if și **operatorul in**, verificați dacă acest produs se află în stoc_total. Dacă da, afișați mesajul "Produsul este disponibil în depozit.".

Pasul 5: Filtrarea erorilor prin non-apartenență (not in) Utilizatorul introduce un alt cod de test: cod_nou = 999. Folosind **operatorul not in**, verificați dacă acest cod lipsește din stoc_total. Dacă lipsește, afișați mesajul de siguranță: "Alertă: Codul introdus nu este valid!".

Pasul 6: Compararea listelor (Audit) Pentru a verifica dacă depozitele sunt identice ca structură și ordine a produselor, comparați depozit_nord cu depozit_sud folosind **operatorul relațional ==**. Afișați direct rezultatul comparării pe ecran.



```
1 depozit_nord = [101, 102, 105]
2 depozit_sud = [201, 204]
3 # Pasul 2: Concatenarea stocurilor# Operatorul + unește listele într-o structură complet nouă, fără a le modifica pe cele inițiale
4 stoc_total = depozit_nord + depozit_sud
5 print("Pasul 2 - Stocul total concatenat:", stoc_total)
6 # Pasul 3: Replicarea / Multiplicarea stocului de urgență
7 # Operatorul * multiplică elementele listei de 3 ori în mod ciclic
8 kit_rezerva = depozit_sud * 3
9 print("Pasul 3 - Kitul de rezervă multiplicat:", kit_rezerva)
10 # Operatorul 'in' verifică prezența unei valori în listă și returnează True/False
11 cod_cautat = 102
12 if cod_cautat in stoc_total:
13     print(f"Pasul 4 - Verificare {cod_cautat}: Produsul este disponibil în depozit.")
14 # Pasul 5: Filtrarea erorilor prin non-apartenență (not in)
15 # Operatorul 'not in' verifică absența unei valori în listă pentru a asigura siguranța datelor
16 cod_nou = 999
17 if cod_nou not in stoc_total:
18     print(f"Pasul 5 - Verificare {cod_nou}: Alertă: Codul introdus nu este valid!")
19 # Operatorul == compară lungimea, elementele și ordinea lor exactă în cele două liste
20 sunt_identice = (depozit_nord == depozit_sud)
21 print("Pasul 6 - Rezultatul auditului (depozite identice):", sunt_identice)
```

Run main ×

```
Pasul 2 - Stocul total concatenat: [101, 102, 105, 201, 204]
Pasul 3 - Kitul de rezervă multiplicat: [201, 204, 201, 204, 201, 204]
Pasul 4 - Verificare 102: Produsul este disponibil în depozit.
Pasul 5 - Verificare 999: Alertă: Codul introdus nu este valid!
Pasul 6 - Rezultatul auditului (depozite identice): False
```

Problemă rezolvată și explicată pas cu pas

Enunț: Se dau două liste ce reprezintă loturile de componente verificate de doi ingineri diferiți: lot_inginer1 = [70, 85] și lot_inginer2 = [70, 85]. Să se scrie un program în limbajul Python care:

1. Concatenează cele două loturi într-o listă extinsă numită lot_total.
2. Generează o listă numită semnal_alerta care multiplică o listă formată dintr-un singur element, [0], de 4 ori.
3. Verifică dacă piesa cu codul 85 aparține listei lot_total.
4. Verifică dacă codul de eroare 99 nu aparține listei lot_total.
5. Compară cele două loturi inițiale pentru a stabili dacă inginerii au scanat aceleași componente în aceeași ordine.

3. Exemplu

- **Date de intrare:** lot_inginer1 = [70, 85], lot_inginer2 = [70, 85]
- **Date de ieșire:**
 - Lot total concatenat: [70, 85, 70, 85]
 - Semnal alertă multiplicat: [0, 0, 0, 0]
 - Rezultat comparare loturi: True

4. Analiza exemplului

- **Operația 1:** Operatorul + combină elementele din lot_inginer1 și lot_inginer2 creând un nou obiect listă: [70, 85, 70, 85].
- **Operația 2:** Operatorul * preia lista [0] și o multiplică matematic în memorie de 4 ori, rezultând lista [0, 0, 0, 0].
- **Operația 3:** Expresia 85 in lot_total analizează elementele. Deoarece 85 se află la indexul 1 și indexul 3, condiția întoarce True.
- **Operația 4:** Expresia 99 not in lot_total verifică absența. Întrucât valoarea 99 chiar nu există în listă, rezultatul evaluat este True.
- **Operația 5:** lot_inginer1 == lot_inginer2 compară lungimea, valorile și pozițiile. Fiind identice, rezultatul final este True.

Implementarea în limbajul Python :

```
lot_inginer1 = [70, 85]
lot_inginer2 = [70, 85]
lot_total = lot_inginer1 + lot_inginer2
semnal_alerta = [0] * 4
if 85 in lot_total:
    print("Componenta 85 este validată.")
if 99 not in lot_total:
    print("Codul de eroare 99 este absent.")
loturi_identice = (lot_inginer1 == lot_inginer2)
print("Lot total:", lot_total)
print("Semnal alertă multiplicat:", semnal_alerta)
print("Verificare loturi identice:", loturi_identice)
```

Testarea și verificarea pas cu pas

Pasul (Instrucțiunea executată)	Starea listei lot_total în memorie	Indecșii pozitivi curepți	Variabile create / Rezultate
Stare inițială	<i>(Nu este creată încă)</i>	<i>(Nu există indecși)</i>	Alocare loturi individuale în memorie
1. L1 + L2	[70, 85, 70, 85]	0: 70, 1: 85, 2: 70, 3: 85	lot_total generat ca obiect nou
2. [0] * 4	[70, 85, 70, 85]	<i>(Nemodificată structural)</i>	semnal_alerta = [0, 0, 0, 0]
3. 85 in lot_total	[70, 85, 70, 85]	0: 70, 1: 85, 2: 70, 3: 85	Evaluat ca fiind True (afișează textul)
4. 99 not in lot_total	[70, 85, 70, 85]	0: 70, 1: 85, 2: 70, 3: 85	Evaluat ca fiind True (afișează textul)
5. L1 == L2	[70, 85, 70, 85]	<i>(Nemodificată structural)</i>	loturi_identice = True

☞ Dezbateri & Justificare tehnică: Claritate și Siguranță în validarea datelor

Atunci când scriem programe profesionale, algoritmii de validare a datelor introduse de utilizatori pot fi scriși în două moduri diferite: fie folosind structuri repetitive clasice (bucle for / while care parcurg fiecare element și îl compară manual), fie folosind direct operatorii de apartenență (in, not in).

De ce utilizarea operatorilor in și not in este considerată o practică optimă și eficientă?

1. **Claritatea Codului (Lizibilitate / Sintaxă curată):** Codul Python devine extrem de apropiat de limbajul natural uman. O instrucțiune de tipul if utilizator not in lista_blocati: este mult mai ușor de înțeles de către orice programator din echipă decât o buclă de 5 linii care numără aparițiile sau folosește un flag de validare boolean. Acest lucru reduce dramatic apariția bug-urilor la scriere.
2. **Siguranță în Prelucrare:** Utilizarea operatorilor dedicați elimină erorile umane clasice asociate parcurgerii listelor, cum ar fi depășirea barierelor de memorie (erori de tip IndexError sau off-by-one errors), deoarece mecanismul intern de căutare este gestionat automat și securizat direct de limbaj.
3. **Eficiență la Rulare:** La nivel intern (în limbajul C în care este scris Python), operatorii in și not in sunt implementați prin iterații optimizate la nivel de mașină, executându-se mult mai rapid decât o parcurgere manuală scrisă de noi într-un script Python standard.

Problemă propusă pentru tema de acasă

🔑 *Scenariu: „Sistemul de Securitate și Validare al unei Aplicații Bancare”*

Context: Dezvoltă modulul de autentificare pentru o aplicație fintech. Serverul de securitate deține o listă cu ID-urile utilizatorilor care au conturile blocate din motive de securitate (utilizatori_blocati), precum și o listă cu ID-urile celor care au raportat tentative de phishing în acea zi (utilizatori_suspecti). Pentru a valida accesul unui client nou, trebuie să prelucrezi aceste liste și să verifici apartenența datelor introduse de la tastatură.

Enunț:

Se dau listele inițiale: utilizatori_blocati = [1001, 1005, 1009] și utilizatori_suspecti = [2002, 2003].

Scrieți un program în limbajul Python care parcurge următoarele cerințe obligatorii:

1. **Centralizarea Listelor de Risc:** Creați o listă unificată complet nouă numită lista_neagra prin combinarea elementelor din cele două liste de mai sus, utilizând **operatorul de concatenare (+)**.
2. **Sistemul de Semnalizare:** În cazul unei breșe de securitate generală, aplicația trebuie să trimită un semnal codificat format dintr-o listă cu 5 valori de zero: [0, 0, 0, 0, 0]. Generați această listă într-o variabilă numită cod_alerta folosind **operatorul de multiplicare (*)** aplicat unei liste inițiale formate dintr-un singur element.
3. **Validarea Accesului (Apartenență):** Utilizatorul introduce ID-ul său pentru autentificare: id_client = 1005. Folosind **operatorul in** aplicat pe lista_neagra, verificați dacă acest ID se află pe lista de risc. Dacă este găsit, afișați textul: "Acces respins! Contul este restricționat.".
4. **Validarea Siguranței (Non-apartenență):** Programul verifică un alt utilizator: id_nou = 5555. Folosind **operatorul not in** aplicat pe lista_neagra, verificați dacă acest ID este sigur (nu se află pe lista neagră). Dacă nu se află, afișați mesajul: "Cont validat. Procedura de autentificare continuă.".
5. **Audit Tehnic:** Comparați structural lista utilizatori_blocati cu lista utilizatori_suspecti folosind **operatorul de inegalitate (!=)** pentru a demonstra că cele două baze de date reflectă probleme de securitate complet diferite. Afișați rezultatul comparației booleene pe ecran.